

```

service_description PING
check_command check_ping!100.0,20%!500.0,60%
}

```

This definition uses a template, *local-service*, defined as:

```

define service{
    name local-service ; The name of this service template
    use generic-service ; Inherit default values from the
generic-service definition
    check_period 24x7 ; The service can be checked at any time of
the day
    max_check_attempts 4 ; Re-check up to 4 times to determine
final (hard) state
    normal_check_interval 5 ; Check service every 5 minutes
normally
    retry_check_interval 1 ; Re-check every minute until a hard
state can be determined
    contact_groups admins ; Send notifications to all in the
'admins' group
    notification_options w,u,c,r ; Send warning, unknown, critical,
and recovery notifications
    notification_interval 60 ; Re-notify about problems every
hour
    notification_period 24x7 ; Notifications can be sent out at
any time
    register 0 ; DONT REGISTER THIS TEMPLATE
DEFINITION!
}

```

The *local-service* template further uses a template, *generic-service*. As mentioned above, we will explore service definition directives in the next article. For our use-case scenario, please ensure that you comment out all other service definitions in this configuration file. We will explore all of them in subsequent articles.

Therefore, to sum up the various files used, based on the default configuration, our Nagios instance is set up thus:

- Will monitor *localhost* (IP address 127.0.0.1)
- It will be monitored 24x7.
- This host is checked using the command `/usr/lib64/nagios/plugins/check_ping`.
- Notifications are sent if the host is down, is unreachable or has recovered.
- Notifications go to *nagios-admin@localhost*, but are sent only during *workhours*, and will be resent every two hours if the host is still down or unreachable.

The CGI configuration file

The CGI configuration file (`/etc/nagios/cgi.cfg`) configures the CGI scripts and the Web GUI of Nagios. The significant parameters are:

- *main_config_file*: The path of the main Nagios

configuration file, and where the CGI scripts should find it.

- *physical_html_path*: The filesystem path for Nagios HTML files.
- *url_html_path*: The URL portion appended to the base URL, that will access the Nagios HTML files.
- *refresh_rate*: Specifies the refresh rate for various CGIs such as status.
- *use_authentication*: Specifies that the CGI scripts should use authentication.

Once Nagios has been configured, you will need to add an authentication file to be able to access Nagios pages. By default, the Apache configuration directives (specified in `/etc/httpd/conf.d/nagios.conf`) rely on basic authentication, and allow access only from *localhost*. The user authentication file `/etc/nagios/passwd` needs to be created. You can do this using the `htpasswd` command:

```
[root@prefect ~]# htpasswd -bc /etc/nagios/passwd nagios-admin admin@123
```

This creates the *nagios-admin* user, with the password *admin@123* and stores the details in the file `/etc/nagios/passwd`.

Hopefully, we are ready to test our base Nagios installation now. Start the Nagios service and check the logs. Restart the Apache service:

```

[root@prefect nagios]# /etc/init.d/httpd restart
[root@prefect nagios]# /etc/init.d/nagios start
[root@prefect nagios]# tailf /var/log/nagios/nagios.log

```

If the Nagios logs are fine, you should now open your browser and connect to `http://localhost/nagios/`, authenticate as *nagios-admin* and check the Host summary. The configured host, *localhost*, should be up.

In the next article, we will explore service definitions, how to define them, and also host-groups and service-groups. We will also monitor services on remote hosts. Do write to me your doubts and queries, if any. I cannot promise an instant reply due to other assignments, but I can assure you that a reply will surely come your way. In any case, there is a wealth of information available on Nagios, and the documentation provided along with the installation is also quite good. Go on, build your prefect and manage your data-centre. **END** 

By: Varad Gupta

Varad is an open source enthusiast who strongly believes in the open source collaborative model not only for technology but also for business. India's first RHCSS (Red Hat Certified Security Specialist), he has been involved in spreading open source through Keen & Able Computers Pvt Ltd, an open source systems integration company, and FOSTERING Linux, a FOSS training, education and research training centre.